Toxic Comment Preprocessing Pipeline

Nov 11, 2026 · @JOOYOUNG SONG

Overview

One file, `preprocess.py`, cleans the Jigsaw comments for both models. Everything is lowercased; we do not use upper/lower case as a signal for now.

There are two branches that share most steps:

Branch	Used by	Steps
`tfidf`	Lexicon / NB-SVM baseline	All steps, including chat spelling
`bert`	BERT (uncased)	Noise tokens, deobfuscation, spam cap only, so the sentence stays as natural as possible

How to run it:

```
pip install pandas scikit-learn rapidfuzz wordfreq iterative-stratification
python preprocess.py --data data/ --out out/
```

It writes `train_clean.csv` and `test_clean.csv` (with `text_tfidf` and `text_bert` columns), `deobf_log.csv` (every word the deobfuscation changed, for checking) and `folds.csv` (a stratified train/validation split).

Step order

The order matters: some steps break each other if they run in the wrong order. For example, leet mapping on an IP address would turn its digits into letters, so IPs are replaced first.

1. **Wikipedia noise** → replace IPs, timestamps, user links and URLs with placeholder words.
2. **Lowercase** everything.
3. **Chat spelling** (`tfidf` branch only): `u` → you, `ur` → your, `r` → are.
4. **Deobfuscation**: turn disguised swear words back into the real word (details below).
5. **Spam cap**: keep at most 5 copies of the same token per comment.
6. **TF-IDF**: fit on the training set only, then transform validation and test.

Tokenizing splits on spaces and peels punctuation off the ends of each word. Symbols *inside* a word (`sh!t`, `f**k`) are kept, because step 4 needs them. A trailing `!` or `?` is kept once as its own token.

Wikipedia noise and chat spelling

We replace Wikipedia noise with placeholder words instead of deleting it, so the model can still see that it was there.

Pattern	Example	Becomes
IP address	`64.56.243.153`	`iptoken`
Timestamp	`05:52, 26 Jun 2005 (UTC)`	`timetoken`
User link	`[[User:Bob|Bob]]`	`usertoken`
URL	`http://...`, `www...`	`urltoken`
Wiki markup	`[[ ]]`, `{{ }}`, `==`, `''`	removed

Anonymous editors sign with an IP address, so `iptoken` may correlate with toxicity. That could be a shortcut the model learns instead of reading the text, so check how much weight it gets.

Chat spelling matters because `you` and `your` are the main signal for insults ("your mom..."). Without it, `u r a idiot` and `you are an idiot` look unrelated. BERT already understands `u`, so this step only runs in the `tfidf` branch.

Modified swear words

The function `deobfuscate()` tries five rules in order and stops at the first one that works. Each word can only become one of the ~20 words in `LEXICON`, and rules a–d only fire when the result is *exactly* a lexicon word.

Rule	Example	Output	Changes on train + test
a. Separators	`f.u.c.k`, `s-h-i-t`	fuck, shit	78 (with b)
b. Repeated letters	`fuckkkk`, `shiiiit`	fuck, shit	(with a)
c. Wildcard	`f*ck`, `b*tch`, `f***ing`	fuck, bitch, fucking	612

Rule	Example	Output	Changes on train + test
d. Leet + sound-alike	`5h1t`, `f@ggot`, `phuck`	shit, faggot, fuck	214
e. Edit distance = 1	`shoit`, `fcuk`, `bicth`	`shoit shit_guess`	575

The rules in detail:

- **a. Separators** are only removed when every piece is 1–2 letters. So `f.u.c.k` is joined, but `re-add` and `u.s` are not.
- **b. Repeated letters**: in the text, 3+ repeats become 2 (`goooood` → `good`). For matching against the lexicon we try both 2 (`asssshole` → asshole) and 1 (`fuckk` → fuck).
- **c. Wildcard**: `*`, `#` and `%` each mean one unknown letter. `f*ck` becomes the pattern `f?ck`, and the word is replaced only if exactly one lexicon word fits. We never map `*` to a fixed letter, because `*`→u would turn `b*tch` into `butch`.
- **d. Leet**: `1 ! → i`, `0 → o`, `3 → e`, `4 @ → a`, `$ 5 → s`, `7 → t`, plus `ph → f`. Only applied to tokens that contain letters, so numbers and years stay intact.
- **e. Edit distance** is the minimum number of single-letter changes (insert, delete, replace a letter, or swap two neighbors) between two words. A guess is made only if all of these hold:
  1. The distance to a lexicon word is exactly 1.
  2. The first letter matches (people keep it so the word stays readable).
  3. The word has at least 4 letters.
  4. It is not an English word (the 100k most common words).
  5. It does not already start with a lexicon word: `dicks`, `fucked`, `idiots` are real inflected forms, not disguises.

Rule e adds a guess instead of replacing. `whome` becomes `whome whore_guess`. The original stays, so a wrong guess only adds a little noise, and the model learns how much to trust `_guess` tokens.

Why we need that: a typo of a normal word looks exactly like a disguised swear word. In the 40 most frequent guesses (394 changes), about 8% were wrong by eye, e.g. `whome`, `cung`, `snit`, `sout`. Rules a–d had no wrong changes in the same check.

One fix worth knowing about: the English word list comes from web text, so it also contains slang like `fuckin`, `biatch` and `fuuuuck`. Words containing a swear stem (`fuc`, `bitch`, `nigg`, ...) are removed from the English list; otherwise those spellings would be

protected from correction.

Every change is written to `deobf_log.csv` (original, new, rule, count). Before training, skim it for wrong guesses.

Stopwords and lemmatization

We do not remove stopwords. The standard NLTK list includes *you, your, I, will, should, not, no*, which are exactly the words insults and threats depend on ("your mom", "I will", "you're not stupid"). TF-IDF already gives very common words a low weight, so removal adds nothing.

We do not lemmatize (reduce words to their base form). Character n-grams already connect `fucking`, `fucked` and `fucker` through shared pieces like `fuc`, and part-of-speech taggers make many mistakes on messy text like this. BERT has its own tokenizer, so it never needs it.

TF-IDF, data split and label issues

TF-IDF is fit on the training set only. `build_tfidf(train_text)` returns two vectorizers; call `.transform()` on validation and test. Fitting on train + test text, as many Kaggle notebooks do, leaks test information into training.

Vectorizer	Units	Setting
word	1–2 words	catches phrases like `your mom`, `not stupid`
char	3–5 characters within words	links spelling variants like `shyt` and `shit`

Both use `sublinear_tf=True`, which replaces the count tf with 1 + log(tf), so a word repeated 300 times does not get 300× the weight.

Split: `folds.csv` assigns each training comment to one of 5 folds with multi-label stratification (`iterstrat`). A random split can leave the rare labels badly underrepresented in validation: threat is only 0.3% of comments.

Duplicates: 801 duplicate comments are merged (159,571 → 158,770 rows). When copies disagree on a label, we keep the label as 1 if any copy has it.

Test set: only the 63,978 test rows with labels are kept. Rows labeled `-1` were not used for scoring in the competition.

NB-SVM on top of TF-IDF

NB-SVM is logistic regression on TF-IDF features that are first re-weighted by how strongly each feature points to a label. Plain LR is TF-IDF → LR; NB-SVM is TF-IDF → scale by r → LR. It is trained separately for each of the 6 labels.

1. **Compute the ratio r for every feature**, using the training comments of that label:

$$r = \log \frac{p / \lVert p \rVert_1}{q / \lVert q \rVert_1}, \qquad p = \alpha + \sum_{i:\, y_i = 1} x_i, \qquad q = \alpha + \sum_{i:\, y_i = 0} x_i$$

Here x_i is comment i's TF-IDF vector, y_i its label (1 or 0), $\alpha = 1$ is smoothing so nothing is divided by zero, and $\|p\|_1$ is the sum of all entries of p. A positive r means the feature is more common in comments with the label; a negative r means it is more common in clean ones.

2. **Scale each feature by its r**: every comment vector x becomes $x \odot r$ (element-wise product).
3. **Train logistic regression** on the scaled features (C = 4, the common default on this competition).

This is the same idea as our lexicon: r is the log ratio of how often a word appears in labeled vs clean comments. The difference is that NB-SVM uses it as a weight inside a classifier instead of a fixed word list. `preds/nb_top_words.csv` shows the words with the highest r per label.

One difference from the original paper (Wang & Manning, 2012): the paper uses binary features (1 if the word appears, else 0), while we compute r from TF-IDF values, the variant that became popular on this Kaggle competition. Binarizing the features first (`X > 0`) would be a small extra comparison.

On the labeled test set, NB-SVM scored 0.9775 mean ROC-AUC vs 0.9797 for plain LR, with C not tuned.

Training and evaluation

The TF-IDF baselines reach 0.980 mean ROC-AUC on the labeled test set, but drop to about 0.88 on comments where an insult word is negated. That gap is what BERT has to close.

File	What it does	Runs on
`preprocess.py`	Cleans text, writes `out/`	CPU, ~1 min

File	What it does	Runs on
`train_tfidf.py`	Logistic regression + NB-SVM, one classifier per label	CPU, ~10 min per run
`train_bert.py`	Fine-tunes BERT with 6 sigmoid outputs and binary cross-entropy	GPU (Colab T4 or better)
`evaluate.py`	ROC-AUC per label + mean, overall and on two subsets	CPU, seconds

Run order:

```
python preprocess.py --data data --out out
python train_tfidf.py --data out --preds preds
python train_bert.py --data out --preds preds
python evaluate.py --pred preds/tfidf_lr_test.csv preds/tfidf_nbsvm_test.csv
preds/bert_test.csv --truth out/test_clean.csv
```

Every model validates on fold 0 and writes `preds/<model>_val.csv` and `preds/<model>_test.csv`, in the same format as Kaggle's sample submission. The TF-IDF models are retrained on all training data before predicting test; BERT uses its best epoch on fold 0.

The evaluation subsets:

- **no_lexicon_word**: comments containing none of ~30 common swear/insult words.
- **negated**: comments where a negation (*not, never, n't, ...*) comes before an insult word in the same clause.
- A label with fewer than 10 positives in a subset is left blank, because its AUC would be too noisy.

Baseline results on the labeled test set (63,978 comments), mean ROC-AUC over the 6 labels:

Subset	Comments	Logistic regression	NB-SVM
All	63,978	0.9797	0.9775
No lexicon word	52,017	0.9702	0.9634
Negated	514	0.8766	0.8771

BERT notes for Colab:

- Switch the runtime to GPU first. `--subset 20000` gives a quick first result before the full run.
- `--model distilbert-base-uncased --max-len 192` is about twice as fast, for a small accuracy cost.
- `train_bert.py` was only tested end to end on CPU with a tiny random model, so expect to fix small things on the first GPU run.

`preds/nb_top_words.csv` lists the 30 word features with the highest NB ratio per label. This is the data-driven version of the lexicon and can go straight into the report.

Known limitations

- **3-letter disguises** like `fuk` are not corrected (rule e needs 4+ letters). Character n-grams still partly cover them.
- **Masked words outside the lexicon**, like `s*n` in `s*n of a b*tch`, stay unchanged. A possible extension is to mask the word and let BERT guess it, keeping only guesses that fit the visible letters.
- **Case is ignored for now.** ALL CAPS and `!!!` likely signal `severe_toxic`; a caps-ratio feature can be added later, taken before lowercasing.
- **Negation** (`you're not stupid`) is not handled in preprocessing. Marking words after a negation with `NOT_` is an optional ablation.
- **The lexicon is hand-picked.** Swapping in a data-driven list (words with the highest toxic-vs-clean ratio) only requires changing `LEXICON`.